

Equivalence of Propositions

Notes By Pr. El Hadiq Zouhair

3.1 Overview

Chapter 2 introduced logical equivalence through truth tables. This chapter turns equivalence into a working calculus: classifying formulas by their satisfiability profile, simplifying them with the standard laws, and putting them into *normal forms* suited to mechanical processing.

3.2 Tautology, contradiction, contingency revisited

Definition 3.1. Let φ be a formula over n atomic letters, with 2^n valuations. φ is a *tautology* if all 2^n valuations satisfy it, a *contradiction* if none does, and a *contingency* otherwise. φ is *satisfiable* when at least one valuation satisfies it — i.e. exactly when it is not a contradiction.

Example 3.2. $(p \rightarrow q) \leftrightarrow (\neg q \rightarrow \neg p)$ is a tautology (the contrapositive law); $(p \oplus q) \wedge (p \leftrightarrow q)$ is a contradiction; $p \rightarrow (q \wedge r)$ is a contingency.

φ is a tautology if and only if $\neg\varphi$ is a contradiction. Tautology checking and satisfiability checking are therefore two faces of the same computational problem.

3.3 The main equivalences

The equivalence laws of Chapter 2 — identity, domination, idempotence, double negation, commutativity, associativity, distributivity, De Morgan, absorption, negation — together with the implication identities

$$p \rightarrow q \equiv \neg p \vee q, \quad \neg(p \rightarrow q) \equiv p \wedge \neg q, \quad p \leftrightarrow q \equiv (p \rightarrow q) \wedge (q \rightarrow p),$$

form a complete working kit. Two further conditional equivalences are worth memorising:

$$p \rightarrow (q \rightarrow r) \equiv (p \wedge q) \rightarrow r \quad (\text{exportation}),$$

$$(p \rightarrow q) \wedge (p \rightarrow r) \equiv p \rightarrow (q \wedge r), \quad (p \rightarrow r) \wedge (q \rightarrow r) \equiv (p \vee q) \rightarrow r.$$

Example 3.3 (Simplifying a proposition). Simplify $\varphi = (p \wedge q) \vee (p \wedge \neg q) \vee (\neg p \wedge q)$.

$$\begin{aligned} \varphi &\equiv (p \wedge (q \vee \neg q)) \vee (\neg p \wedge q) && \text{distributivity} \\ &\equiv p \vee (\neg p \wedge q) && \text{negation, identity} \\ &\equiv (p \vee \neg p) \wedge (p \vee q) && \text{distributivity} \\ &\equiv p \vee q && \text{negation, identity.} \end{aligned}$$

Three two-literal clauses collapse to a single disjunction.

Such simplification is exactly what automated tools (SAT preprocessors, circuit minimisers, symbolic algebra systems) perform on a large scale; the Quine–McCluskey algorithm and Karnaugh maps systematise it for circuit design.

3.4 Disjunctive normal form

Definition 3.4. A *minterm* over atoms $p_1, \dots, p_n$ is a conjunction containing each p_i exactly once, either plain or negated. A formula is in *disjunctive normal form* (DNF) if it is a disjunction of conjunctions of literals; it is in *full* (canonical) DNF if every disjunct is a minterm.

Theorem 3.5 (DNF from the truth table). Every satisfiable formula φ is equivalent to the disjunction of the minterms of the rows where φ is true. Hence every formula has a full DNF, unique up to order.

Example 3.6 (XOR: “for me or against me”). The exclusive or is true on rows (T, F) and (F, T), so its full DNF is

$$p \oplus q \equiv (p \wedge \neg q) \vee (\neg p \wedge q).$$

Example 3.7 (A counting function). Define $m(p, q, r)$ to be true when *at least two* of p, q, r are true (the “majority” function). The true rows are (T, T, T), (T, T, F), (T, F, T), (F, T, T), giving the full DNF

$$m \equiv (p \wedge q \wedge r) \vee (p \wedge q \wedge \neg r) \vee (p \wedge \neg q \wedge r) \vee (\neg p \wedge q \wedge r),$$

which simplifies (by absorption-style grouping) to $(p \wedge q) \vee (p \wedge r) \vee (q \wedge r)$.

3.5 Other normal forms

Definition 3.8. A formula is in *negation normal form* (NNF) if negation appears only on atoms, and in *conjunctive normal form* (CNF) if it is a conjunction of *clauses*, each clause a disjunction of literals. The full CNF uses *maxterms* — clauses containing every atom once — one for each *false* row of the truth table.

CNF is the input format of SAT solvers and of the resolution proof system; DNF makes satisfiability trivial to read off but may be exponentially large.

3.5.1 CNF $\leftrightarrow$ DNF conversion

The two directions are symmetric: distribute $\vee$ over $\wedge$ to go from CNF to DNF, and $\wedge$ over $\vee$ for the converse.

Example 3.9.

$$(p \vee q) \wedge (\neg p \vee r) \equiv (p \wedge \neg p) \vee (p \wedge r) \vee (q \wedge \neg p) \vee (q \wedge r) \equiv (p \wedge r) \vee (\neg p \wedge q) \vee (q \wedge r),$$

a CNF of two clauses becoming a DNF of three conjunctions (the contradictory term vanishes). The term $q \wedge r$ is in fact absorbed by no other and must stay; in general the blow-up can be exponential.

3.6 Satisfiability research

Deciding satisfiability of a CNF formula is the historic NP-complete problem (Cook–Levin, 1971). Despite the exponential worst case, modern conflict-driven clause-learning (CDCL) solvers routinely handle industrial instances with millions of variables, powering hardware verification, planning and dependency resolution. The special case 2-SAT (clauses of width 2) is solvable in linear time, while 3-SAT is already NP-complete — a remarkably sharp frontier.

3.7 Summary

Formulas split into tautologies, contradictions and contingencies; the equivalence laws constitute an algebra for simplifying them; every formula has a CNF and a DNF, readable from the false and true rows of its truth table respectively; and satisfiability, trivial for DNF, is the canonical hard problem for CNF.

Exercises

Exercise 3.1 EASY

Classify as tautology, contradiction or contingency: (a) $(p \wedge q) \rightarrow p$; (b) $p \leftrightarrow \neg p$; (c) $(p \vee q) \wedge \neg p$.

Exercise 3.2 EASY

Verify by truth table the exportation law $p \rightarrow (q \rightarrow r) \equiv (p \wedge q) \rightarrow r$.

Exercise 3.3 MEDIUM

Simplify using the laws, citing each step: $\neg(\neg p \wedge \neg q) \wedge (p \vee \neg q)$.

Exercise 3.4 MEDIUM

Simplify $(p \rightarrow q) \wedge (p \rightarrow \neg q)$ to a formula with a single connective.

Exercise 3.5 MEDIUM

Write the full DNF of $p \rightarrow \neg q$ (two atoms), directly from its truth table.

Exercise 3.6 MEDIUM

Give the full CNF of $p \oplus q$, using the false rows of its truth table.

Exercise 3.7 HARD

Let $f(p, q, r)$ be true exactly when an *odd* number of p, q, r are true. Write its full DNF and show $f \equiv p \oplus q \oplus r$.

Exercise 3.8 HARD

Convert $(p \wedge q) \vee (r \wedge s)$ into CNF, and explain why a CNF of a DNF with k conjunctions of two literals each may need 2^k clauses.

Exercise 3.9 HARD

Show that the majority function $m \equiv (p \wedge q) \vee (p \wedge r) \vee (q \wedge r)$ is also equivalent to the CNF $(p \vee q) \wedge (p \vee r) \wedge (q \vee r)$.

Solutions to the Exercises

Solution 3.1. (a) Tautology — if $p \wedge q$ holds, p holds. (b) Contradiction — the two sides always differ. (c) Contingency — true for $p = \text{F}, q = \text{T}$, false for $p = \text{T}$. ■

Solution 3.2. Both sides are false exactly on the row $(p, q, r) = (\text{T}, \text{T}, \text{F})$: the left side because $q \rightarrow r$ then fails with p true; the right side because $p \wedge q$ is true and r false. All other rows make both sides true, hence equivalence. ■

Solution 3.3.

$$\begin{aligned}
 \neg(\neg p \wedge \neg q) \wedge (p \vee \neg q) &\equiv (p \vee q) \wedge (p \vee \neg q) && \text{De Morgan, double negation} \\
 &\equiv p \vee (q \wedge \neg q) && \text{distributivity} \\
 &\equiv p \vee \text{F} \equiv p && \text{negation, identity.}
 \end{aligned}$$

■

Solution 3.4. $(p \rightarrow q) \wedge (p \rightarrow \neg q) \equiv p \rightarrow (q \wedge \neg q) \equiv p \rightarrow \text{F} \equiv \neg p$. ■

Solution 3.5. $p \rightarrow \neg q$ is true on rows $(\text{T}, \text{F}), (\text{F}, \text{T}), (\text{F}, \text{F})$, so

$$p \rightarrow \neg q \equiv (p \wedge \neg q) \vee (\neg p \wedge q) \vee (\neg p \wedge \neg q).$$

■

Solution 3.6. $p \oplus q$ is false on rows (T, T) and (F, F) . Each false row yields the maxterm that negates it: $(\neg p \vee \neg q)$ for (T, T) and $(p \vee q)$ for (F, F) . Hence $p \oplus q \equiv (p \vee q) \wedge (\neg p \vee \neg q)$. ■

Solution 3.7. Odd-parity rows: exactly one true — $(\text{T}, \text{F}, \text{F}), (\text{F}, \text{T}, \text{F}), (\text{F}, \text{F}, \text{T})$ — or all three true $(\text{T}, \text{T}, \text{T})$. Full DNF:

$$(p \wedge \neg q \wedge \neg r) \vee (\neg p \wedge q \wedge \neg r) \vee (\neg p \wedge \neg q \wedge r) \vee (p \wedge q \wedge r).$$

Since $\oplus$ is associative and $x \oplus y$ is true when exactly one of x, y is true, $(p \oplus q) \oplus r$ is true when the number of true atoms among p, q, r is odd — the same four rows. Hence $f \equiv p \oplus q \oplus r$. ■

Solution 3.8. Distribute:

$$(p \wedge q) \vee (r \wedge s) \equiv (p \vee r) \wedge (p \vee s) \wedge (q \vee r) \wedge (q \vee s).$$

With k two-literal conjunctions $(a_1 \wedge b_1) \vee \cdots \vee (a_k \wedge b_k)$, distributing chooses one literal from each conjunct in every clause: 2^k clauses, and in general (for distinct variables) none is redundant. ■

Solution 3.9. Expand the CNF: $(p \vee q) \wedge (p \vee r) \wedge (q \vee r)$. If at least two of p, q, r are true, each clause contains a true literal, so the CNF is true. If at most one is true, some pair among $\{p, q\}, \{p, r\}, \{q, r\}$ consists of two false atoms, and the corresponding clause fails. So the CNF is true exactly when at least two atoms are true — the majority function m . ■